

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Дисциплина «Бекэнд-разработка»



ОТЧЕТ по

лабораторной работе 2

“Межсервисное взаимодействие через RabbitMQ”

Выполнил:

Таипов Т.А.

Группа:

К3441

Преподаватель:

Добряков Д.И.

Санкт-Петербург

2026 г.

Содержание

ВВЕДЕНИЕ	2
1 Задача	2
2 Ход работы	2
2.1 Краткое описание предметной области	2
2.2 Цели внедрения брокера сообщений	2
2.3 Подключение RabbitMQ	2
2.4 Архитектурная схема обмена	3
2.5 Реализация producer в Application Service	3
2.6 Формат событий и данные сообщения	3
2.7 Реализация consumer в Job Service	4
2.8 Гарантии доставки и обработка ошибок	4
2.9 Дополнительная настройка сетевого взаимодействия	4
2.10 Маршрутизация и расширяемость	4
2.11 Схема взаимодействия	5
2.12 Тестовый сценарий проверки	5
2.13 Проблемы и решения	5
2.14 Риски и меры предотвращения	5
2.15 Перспективы развития	6
2.16 Сравнение RabbitMQ и Kafka	6
2.17 Нефункциональные требования	6
2.18 Варианты расширения сценариев	6
2.19 Оценка нагрузки и пропускной способности	7
2.20 Требования к идемпотентности	7
2.21 Мониторинг и логирование	7
2.22 Безопасность и конфигурация	7
2.23 Итоговая оценка эффекта	7
3 Вывод	8

ВВЕДЕНИЕ

В лабораторной работе реализовано асинхронное межсервисное взаимодействие в микросервисном приложении `jobboard-microservices` с использованием RabbitMQ.

1 ЗАДАЧА

Текст задачи на лабораторную/практическую работу:

- подключить и настроить RabbitMQ/Kafka;
- реализовать межсервисное взаимодействие посредством RabbitMQ/Kafka.

2 ХОД РАБОТЫ

2.1 Краткое описание предметной области

Система представляет собой платформу для поиска работы, где кандидаты откликаются на вакансии работодателей. Для масштабирования и повышения надежности взаимодействия между микросервисами перенесено на асинхронный обмен событиями через брокер сообщений.

2.2 Цели внедрения брокера сообщений

Выбор RabbitMQ обусловлен следующими целями:

- **Асинхронность** между сервисами и снижение времени ответа;
- **Ослабление связности** между производителем и потребителем;
- **Буферизация** событий при пиковых нагрузках;
- **Гарантии доставки** с подтверждениями сообщений.

2.3 Подключение RabbitMQ

В качестве брокера сообщений выбран RabbitMQ. В файл `jobboard-microservices/docker-compose.yml` добавлен сервис:

- `rabbitmq:3-management`;
- порт 5672 для AMQP;
- порт 15672 для web-интерфейса управления;
- подключение к общей сети `jobboard-network`.

Для сервисов `application-service` и `job-service` добавлены переменные окружения:

- `RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672`;

- `RABBITMQ_EXCHANGE=jobboard.events`;
- для `job-service` дополнительно очередь и routing key: `RABBITMQ_APPLICATION_QUEUE` и `RABBITMQ_APPLICATION_ROUTING_KEY`.

2.4 Архитектурная схема обмена

В системе реализована модель “producer–broker–consumer”:

- **Producer:** `Application Service`;
- **Broker:** `RabbitMQ` (exchange `jobboard.events`);
- **Consumer:** `Job Service` (очередь `job-service.application-submitted`).

Exchange выбран типа `topic`, что позволяет в дальнейшем расширять схему событий без изменения инфраструктуры.

2.5 Реализация producer в Application Service

В `application-service` добавлен модуль `src/messaging/rabbit.ts`, который:

- устанавливает соединение с `RabbitMQ`;
- выполняет `assertExchange` для `jobboard.events` (тип `topic`);
- публикует сообщения через функцию `publishEvent(...)`.

После успешного создания отклика (`POST /api/applications`) сервис публикует событие `application.submitted` с данными:

- `id` отклика;
- `id` вакансии;
- `id` кандидата и профиля;
- статус и время создания.

2.6 Формат событий и данные сообщения

Событие `application.submitted` содержит минимальный набор данных для реакции потребителя:

```
{
  "application_id": 101,
  "job_id": 77,
  "candidate_profile_id": 12,
  "candidate_user_id": 5,
  "status": "submitted",
```

```
"created_at": "2026-02-09T12:34:56.000Z"
}
```

Такой формат позволяет сервису-получателю логировать событие и при необходимости выполнять дополнительные действия (уведомления, аналитика).

2.7 Реализация consumer в Job Service

В `job-service` добавлен модуль `src/messaging/rabbit.ts`, который:

- подключается к RabbitMQ;
- создает/проверяет `exchange jobboard.events`;
- создает очередь `job-service.application-submitted`;
- связывает очередь с ключом `application.submitted`;
- запускает consumer с подтверждением сообщений (`ack`).

Запуск consumer интегрирован в `job-service/src/server.ts` при старте сервиса. При получении события сервис пишет в лог информацию об отклике и вакансии.

2.8 Гарантии доставки и обработка ошибок

Для обеспечения надежности используются следующие механизмы:

- **durable exchange/queue** — сохранение метаданных после перезапуска;
- **persistent messages** — сообщения не теряются при рестарте брокера;
- **ACK** — подтверждение успешной обработки сообщения;
- **NACK** — отклонение сообщения при ошибках десериализации.

В случае сбоя consumer повторный запуск приводит к продолжению обработки накопленных сообщений.

2.9 Дополнительная настройка сетевого взаимодействия

Для корректного обращения `application-service` к `job-service` в Docker-сети проверка вакансии переведена на адрес по имени сервиса:

- `JOB_SERVICE_URL=http://job-service:4002`.

Это исключает зависимость от `localhost` внутри контейнера.

2.10 Маршрутизация и расширяемость

Использование `topic exchange` позволяет внедрять новые события, например:

- `job.published` — уведомление о публикации вакансии;
- `application.status.changed` — изменение статуса отклика;

— `message.sent` — новые сообщения в чате отклика.

Каждый сервис может подписаться только на интересующие события, сохраняя низкую связанность.

2.11 Схема взаимодействия

Client -> API Gateway -> Application Service

Application Service -> PostgreSQL (создание отклика)

Application Service -> RabbitMQ Exchange (`application.submitted`)

RabbitMQ Queue(`job-service.application-submitted`) -> Job Service Consumer

Job Service -> лог обработки события

2.12 Тестовый сценарий проверки

Для проверки работы очереди выполнены следующие шаги:

1. запуск инфраструктуры: PostgreSQL, RabbitMQ, все микросервисы;
2. регистрация кандидата и получение JWT;
3. создание вакансии работодателем;
4. отправка отклика кандидатом через `POST /applications`;
5. проверка логов `job-service` на получение события.

Получение события в `consumer` подтверждает корректную настройку брокера и маршрутизации.

2.13 Проблемы и решения

В ходе реализации были учтены следующие моменты:

- обработка временной недоступности RabbitMQ решается повторными попытками подключения;
- использование DNS-имени контейнера гарантирует корректное соединение в Docker-сети;
- единый exchange упрощает управление событиями и настройку подписок.

2.14 Риски и меры предотвращения

Основные риски при работе с брокером сообщений:

- **потеря сообщений** — предотвращается настройкой `durable + persistent`;
- **зависание consumer** — решается таймаутами и мониторингом;
- **дублирование обработки** — требует идемпотентности на стороне `consumer`.

2.15 Перспективы развития

В дальнейшем возможно:

- добавление отдельных очередей для уведомлений (email/SMS);
- внедрение Dead Letter Queue для “плохих” сообщений;
- использование метрик RabbitMQ для мониторинга нагрузки;
- миграция на Kafka для высоконагруженных сценариев.

2.16 Сравнение RabbitMQ и Kafka

Для учебной задачи выбран RabbitMQ, однако при проектировании важно понимать разницу между брокерами:

- **RabbitMQ** хорошо подходит для команд и систем, где важны подтверждения доставки, очереди и сложная маршрутизация (topic/direct/fanout).
- **Kafka** ориентирована на обработку больших потоков событий и хранение логов, где важна высокая пропускная способность и возможность повторной обработки.
- RabbitMQ проще в настройке для небольших проектов, Kafka требует более сложной инфраструктуры (zookeeper/raft).

Таким образом, выбор RabbitMQ оправдан для проекта средней сложности и хорошо демонстрирует принципы событийного взаимодействия.

2.17 Нефункциональные требования

Для устойчивой работы обмена сообщениями следует учитывать нефункциональные требования:

- **Производительность** — брокер должен обрабатывать сотни/тысячи событий без деградации;
- **Надежность** — потеря сообщений недопустима при сбоях, поэтому используются durable/persistent;
- **Масштабируемость** — потребители могут масштабироваться горизонтально;
- **Наблюдаемость** — важно иметь метрики очередей и мониторинг задержек доставки.

2.18 Варианты расширения сценариев

Событийная модель позволяет гибко расширять функциональность:

1. **Уведомления работодателя:** при новом отклике публиковать событие и отправлять email/SMS.

2. **Автоматическая модерация:** подключить сервис фильтрации откликов по ключевым словам.
3. **Аналитика:** подписаться на события и строить отчеты по конверсии кандидатов.

2.19 Оценка нагрузки и пропускной способности

Для оценки производительности рекомендуется зафиксировать целевые показатели:

- среднее время доставки события (end-to-end latency);
- количество сообщений в секунду (throughput);
- рост очереди при пиковых нагрузках.

Даже базовая проверка с нагрузочными запросами к `/applications` помогает выявить узкие места: медленные consumer-ы, ограничение ресурсов контейнеров или неверные настройки `prefetch`.

2.20 Требования к идемпотентности

При повторной доставке сообщений (повторная отправка после NACK или сетевых сбоев) consumer должен уметь безопасно обрабатывать дубли. Практически это означает:

- хранение `id` обработанных событий;
- проверку наличия записи перед повторным созданием;
- логирование повторов без нарушения целостности данных.

2.21 Мониторинг и логирование

Для контроля очередей и состояния брокера рекомендуется:

- использовать web-интерфейс RabbitMQ (<http://localhost:15672>);
- отслеживать метрики: размер очереди, количество consumer-ов, скорость доставки;
- централизовать логи сервисов для отслеживания цепочек событий.

2.22 Безопасность и конфигурация

В учебной среде используются стандартные учетные данные `guest/guest`. В промышленной среде необходимо:

- создавать отдельные учётные записи и ограничивать права;
- хранить ключи в переменных окружения или секрет-хранилищах;
- ограничивать доступ к порту управления RabbitMQ.

2.23 Итоговая оценка эффекта

Внедрение брокера сообщений позволило:

- снизить зависимость между сервисами;
- обеспечить асинхронную передачу событий;
- подготовить архитектуру к дальнейшему масштабированию.

3 ВЫВОД

В ходе лабораторной работы подключен и настроен RabbitMQ, а также реализовано межсервисное взаимодействие по событийной модели: `application-service` публикует событие `application.submitted`, `job-service` принимает и обрабатывает его через очередь RabbitMQ. Задача лабораторной работы выполнена.